AUTOMATED SECURITY ASSESSMENT & REPORTING PIPELINE

รายงานการแก้ไขระบบ ขั้นที่ 6

หน้าเว็บ React + TypeScript · Dashboard · สั่งสแกนจากเว็บ · ดาวน์โหลด/พิมพ์รายงาน
CI/CD · security gate · แก้รหัสผ่านหลุด · ให้ระบบสแกนตัวเอง
ต่อจาก "รายงานการแก้ไขระบบ ขั้นที่ 5"

> ชื่อระบบ: **VulnCompass**
> โปรเจกต์: `C:\Allprojects\Projects`
> GitHub: `github.com/jjjecop/security-pipe-ine` (private)
> ช่วงเวลา: 14–16 ตุลาคม 2026 · 20 commit + PR #1 · test 83 → **102** ตัว

1. สรุปสั้นๆ

ขั้นที่ 5 ทำให้ระบบ**ถูกเรียกใช้จากภายนอกได้อย่างปลอดภัย** (API, คิว, login) ขั้นที่ 6
ทำให้ระบบ**มีหน้าเว็บที่คนใช้งานได้จริง** และ**ทำงานใน CI/CD ได้** ครบเป้าหมาย 3 ทางที่ตั้งไว้: CLI, CI/CD และ web app
ระหว่างทางยังเจอและแก้**รหัสผ่านฐานข้อมูลที่หลุดอยู่ใน git** ซึ่งเป็นปัญหาด้าน security ที่ร้ายแรงที่สุดที่เจอในโปรเจกต์นี้

เรื่อง	ก่อน (จบขั้นที่ 5)	หลัง (จบขั้นที่ 6)
ดูผลสแกน	ผ่าน API ที่หน้า `/docs` หรือเปิด PDF	หน้าเว็บ: การ์ด 5 ระดับ, กราฟแยกตามเครื่องมือ, ตารางค้นหา/กรอง/แบ่งหน้า
สั่งสแกน	`POST /scans` แล้วถามสถานะเอง	ฟอร์มบนเว็บ สถานะเปลี่ยนเองทุก 2 วินาที ตารางประวัติโหลดใหม่เมื่อเสร็จ
รายงาน	ดาวน์โหลดผ่าน API (ต้องมี key)	ปุ่ม **ดาวน์โหลด** และ **เปิดดู/พิมพ์** ในตารางประวัติ
CI	รันแค่ `pytest`	pytest + frontend (lint, build) + **VulnCompass สแกนตัวเอง** ทุก push/PR
ตัดสินผ่าน/ไม่ผ่าน	ไม่มี	`--fail-on critical` จบด้วย exit code 1 เมื่อเจอช่องโหว่ร้ายแรง
รหัสผ่านฐานข้อมูล	**อยู่ใน alembic.ini บน GitHub**	อ่านจาก `.env` เท่านั้น และ**เปลี่ยนรหัสใหม่แล้ว**
Postgres / Redis	เปิดรับทุกเครื่องในเครือข่าย	รับเฉพาะ `127.0.0.1`
action ใน CI	อ้างด้วย tag `@v4` (ถูกย้ายได้)	ล็อกด้วย **commit SHA**, Node 24, Ubuntu 24.04
สร้างฐานข้อมูลใหม่	ทำไม่ได้ (migration แรกว่างเปล่า)	`python init_db.py`
สถานะ "เสร็จ"	ขึ้นก่อนสร้างรายงานเสร็จ (race condition)	ขึ้นหลังรายงานพร้อมแล้วเท่านั้น
จำนวน test	83	102

ภาพรวมสถาปัตยกรรมตอนนี้

```
Browser (React + TypeScript, localhost:5173)
  Login | Dashboard | การสแกน
     |  fetch('/api/...')   cookie HttpOnly แนบให้อัตโนมัติ
     v
Vite dev server  -- proxy: ตัด /api ออก -->  FastAPI main.py (127.0.0.1:8000)
                                              require_auth: session หรือ X-API-Key
                                               |                    |
                      GET summary/findings/scan-jobs     POST /scans -> RPUSH
                                               |                    |
                                               v                    v
                                        PostgreSQL  <--------  worker.py
                                       (127.0.0.1)   run_full_pipeline()
                                                     scan -> CWE -> EPSS -> report -> status

GitHub (ทุก push / PR)
  Tests:            pytest | frontend: npm ci --ignore-scripts -> lint -> build
  Security self-scan: Postgres ชั่วคราว -> init_db -> Semgrep + Trivy -> --fail-on critical ->
PDF
```

2. C5.1–C5.2: เริ่มโปรเจกต์ React และหน้า login

1 C5.1 ติดตั้งและสร้างโปรเจกต์

เครื่องมือ

เครื่องมือ	คืออะไร (เทียบกับฝั่ง Python)
Node.js 24 + npm	ตัวรัน JavaScript และตัวติดตั้ง library (เทียบได้กับ Python + pip)
Vite	เครื่องมือสร้างโปรเจกต์และ dev server แก้โค้ดแล้วหน้าเว็บเปลี่ยนทันทีโดยไม่ต้อง refresh (**HMR**)
TypeScript	JavaScript ที่มีชนิดข้อมูล ตรวจ bug ได้ก่อนรัน (คล้าย type hint + mypy แต่บังคับจริง)
ESLint	linter: อ่านโค้ดแล้วเตือนจุดที่น่าจะเป็น bug เช่นใช้ React hook ผิดวิธี เลือกแทน Oxlint เพราะเป็นมาตรฐานของวงการ
`package.json` / `package-lock.json`	รายชื่อ library / เวอร์ชันที่ล็อกไว้ทุกตัว (เหมือน `requirements.txt` ที่ pin ครบ)
`node_modules/`	library ที่ติดตั้งแล้ว หลายหมื่นไฟล์ ห้าม commit (เหมือน `venv/`)

Vite proxy ตัดสินใจสำคัญ

ใน `vite.config.ts` ตั้งให้ `/api/...` ส่งต่อไปที่ `http://127.0.0.1:8000/...` โดยตัด `/api` ออก ทำให้ browser เห็นหน้าเว็บและ API เป็น**เว็บเดียวกัน** (localhost:5173) จึงไม่ต้องเปิด CORS และ cookie `SameSite=Strict` ที่ทำไว้ใน C4 ใช้ได้ทันที

ก่อน commit

`git add --dry-run frontend` ซ้อมดูว่าจะ add อะไร ต้องไม่มี `node_modules` แม้แต่บรรทัดเดียว (Vite ใส่ไว้ใน `.gitignore` ให้แล้ว)

ไฟล์: `frontend/` ทั้งโฟลเดอร์ (19 ไฟล์), `vite.config.ts`

2 C5.2 หน้า login และการเช็ก session

`api.ts`

`apiFetch<T>()` เรียก API ทุกครั้งผ่านฟังก์ชันเดียว แปลง error เป็น `ApiError` ที่มี `status` และข้อความ `detail` จาก FastAPI · `<T>` คือ generic บอกว่าข้อมูลที่ได้กลับมาเป็นชนิดอะไร

3 สถานะใน `App.tsx`

`undefined` = กำลังเช็ก (แสดง "กำลังโหลด") · `null` = ยังไม่ login (แสดงหน้า login) · `string` = login แล้ว ถาม `/auth/me` ตอนเปิดหน้า ทำให้กด F5 แล้วยัง login อยู่

รายละเอียดในฟอร์ม

`autoComplete="username"` / `"current-password"` ให้ password manager ทำงานถูก · ปุ่ม disabled ระหว่างส่ง กันกดซ้ำ · ใส่รหัสผิดแล้วล้างช่องรหัส

ยืนยัน HttpOnly

F12 → Console → `document.cookie` ได้ `''` แปลว่า JavaScript อ่าน cookie ไม่ได้ ถ้าเว็บโดน XSS ก็ขโมย session ไม่ได้

ไฟล์: `api.ts`, `LoginPage.tsx` (ใหม่), `App.tsx`, `App.css`, `index.css`

> **ทำไม `/health` ต้องเช็กฐานข้อมูลด้วย:** ครั้งแรกที่หน้าเว็บต่อ API ได้ กลับขึ้น "ต่อฐานข้อมูลไม่ได้" เพราะ Docker Desktop ปิดอยู่ ถ้า `/health` เช็กแค่ว่า API ยังรันอยู่ จะตอบ `ok` ทั้งที่ระบบใช้งานจริงไม่ได้

3. C5.3: หน้า Dashboard

3 C5.3a `GET /projects/{name}/summary`

ได้อะไร

`open_total`, `by_severity` (มีครบ 5 ระดับเสมอ แม้บางระดับเป็น 0 หน้าเว็บจะได้ไม่ต้องเช็กว่ามี key หรือไม่), `by_tool`, `last_scan`

ทำไมใช้ `GROUP BY`

ให้ฐานข้อมูลนับใน query เดียว แทนการดึง 336 แถวมานับใน Python

bug จากการวางโค้ด test ผ่านทั้งที่โค้ดพัง

ตอนวางโค้ด บรรทัด `SEVERITY_ORDER = (...)` หายไป (→ `NameError`, HTTP 500) และ `@app.get("/scan-jobs/{job_id}")` หลุดไปอยู่บนฟังก์ชัน summary ทำให้ `/scan-jobs/{id}` พังไปด้วย test ทุกตัวยังผ่าน เพราะมีแค่ test ที่เช็กว่า "ไม่ login ต้องถูกปฏิเสธ" ซึ่งถูกปฏิเสธก่อนถึงโค้ดที่พัง

ไฟล์: `main.py`, `tests/test_api_auth.py`

4 C5.3b การ์ดสรุป + เลือกโปรเจกต์

`types.ts`

บอก TypeScript ว่าข้อมูลจาก API หน้าตาเป็นอย่างไร (`Project`, `Summary`, ...) · `SEVERITIES = [...] as const` แล้วสร้าง type จากลิสต์นั้น ทำให้ลิสต์กับ type ไม่มีวันไม่ตรงกัน

วันที่

ฐานข้อมูลเก็บเวลาแบบ UTC ที่ไม่มีเขตเวลากำกับ จึงต่อ `'Z'` ก่อนแปลง แล้วแสดงด้วย `th-TH` (ปี พ.ศ. เช่น 15/10/2569 14:40:21)

สีของระดับความรุนแรง

ใช้แถบสีด้านซ้ายคู่กับคำว่า วิกฤต/สูง/ปานกลาง/ต่ำ/ข้อมูล ไม่ใช้สีอย่างเดียว (คนตาบอดสีอ่านได้)

ไฟล์: `types.ts`, `Dashboard.tsx` (ใหม่), `App.tsx`, `App.css`

5 C5.3c กราฟแยกตามเครื่องมือ (Recharts)

ตัดสินใจ

`npm install recharts@3.10.1 --save-exact` ล็อกเวอร์ชันเป๊ะ และเลือกเวอร์ชันที่ออกมาเกิน 2 สัปดาห์ (ถ้าเวอร์ชันใหม่มีปัญหาหรือถูกแฮก ส่วนใหญ่มีคนเจอในช่วงนั้น) · กราฟแท่งแนวนอนสีเดียว เรียงจากมากไปน้อย มีตัวเลขท้ายแท่ง เพราะข้อมูลเป็นหมวดหมู่ ไม่ได้ต้องการแยกสีแต่ละแท่ง

ไฟล์: `ToolChart.tsx` (ใหม่), `package.json`, `package-lock.json`

6 C5.3d1 ค้นหาด้วย `?q=` กัน wildcard injection

ทำงานยังไง

`ILIKE '%คำค้น%'` (ไม่สนตัวพิมพ์ใหญ่เล็ก) ใน title, rule id, CWE และ location (แปลง JSON เป็นข้อความด้วย `cast`)

ปัญหา

ใน LIKE ตัว `%` = อะไรก็ได้ และ `_` = ตัวอักษรใดก็ได้ 1 ตัว ถ้าผู้ใช้พิมพ์ `%` จะได้ทุกแถว และค้นคำที่มี `_` จริงๆ ไม่ได้

แก้ด้วย `like_pattern()`

ใส่ `\` หน้าอักขระพิเศษ **ลำดับสำคัญ**: ต้อง escape `\` ก่อน แล้วค่อย `%` และ `_` ไม่อย่างนั้น `\` ที่เพิ่งใส่จะถูก escape ซ้ำ · `Query(None, max_length=100)` กันข้อความยาวเกิน · ยังใช้ parameter ของ SQLAlchemy เสมอ ไม่ต่อ string เข้า SQL (กัน SQL injection)

ข้อจำกัดที่รู้

ค้น `_` แล้วได้ทั้ง 336 แถว เพราะชื่อ key ใน JSON ของ location มี `_` (เช่น `file_path`) แก้ได้ด้วย full-text search ใน C7

ไฟล์: `main.py`, `tests/test_search.py` (ใหม่, 3 test)

7 C5.3d2 ตารางซ่องโหว่

เทคนิค

ส่วน	ทำไม
debounce 300 ms	รอให้หยุดพิมพ์ก่อนค่อยค้น ไม่ยิง API ทุกตัวอักษร
`URLSearchParams`	สร้าง query string ที่ encode อักขระพิเศษถูกต้อง ไม่ต่อ string เอง
`ignore` flag ใน effect	กันผลของ request เก่าที่ตอบกลับช้ามาทับผลของ request ใหม่
`key={summary.project}`	เปลี่ยนโปรเจกต์แล้ว React สร้างตารางใหม่ ตัวกรองและหน้าเริ่มต้นใหม่หมด
discriminated union	`location` มี 4 แบบ (code, dependency, container, http) TypeScript บังคับให้จัดการครบทุกแบบ
แบ่งหน้า 20 แถว	ใช้ `limit` / `offset` ที่ API มีอยู่แล้ว

แก้ CSS

สีของ badge ไม่ขึ้น เพราะ `border-left: 4px solid` ใน `.badge` ทับสีของ `.severity-*` แยกเป็น `border-left-width` + `border-left-style` แล้วสีขึ้น

ไฟล์: `FindingsTable.tsx` (ใหม่), `types.ts`, `Dashboard.tsx`, `App.css`

4. C5.4: หน้าการสแกน

8 C5.4a `GET /projects/{name}/scan-jobs` + test จับ decorator หลุด

endpoint

ประวัติงานล่าสุด (`limit` 1–100 ค่าเริ่ม 20) พร้อม `has_report` ที่ดึงด้วย **query เดียว** (`IN (...)`) แทนการถามทีละงาน 20 ครั้ง ปัญหาแบบหลังเรียก **N+1 query**

`tests/test_routes.py`

decorator หลุดจาก `get_scan_job` เป็นครั้งที่ 2 จึงเขียน test ที่เช็กว่าแต่ละ URL ชี้ไปที่ฟังก์ชันของตัวเองจริง ลองเอา decorator ออกแล้ว test พังจริง (พิสูจน์ว่า test จับบั๊กนี้ได้)

ไฟล์: `main.py` , `tests/test_routes.py` (ใหม่), `tests/test_api_auth.py`

9 C5.4b React Router + C5.4c หน้าประวัติ

Router

`react-router@7.18.4` (ไม่เลือก v8 เพราะใหม่เกินไป) · `NavLink` ไฮไลต์เมนูที่อยู่ · `end` บน `/` ไม่ให้ Dashboard ไฮไลต์ตอนอยู่หน้า `/scans` · `path="*"` แสดง "ไม่พบหน้านี้"

โค้ดที่ใช้ร่วมกัน

ย้าย `formatDate` และ `errorMessage` ไป `format.ts` ให้ทั้ง 2 หน้าใช้ร่วมกัน

สถานะ

`⌛ รอคิว · ⟳ กำลังสแกน · ✓ เสร็จ · ⚠ เสร็จบางส่วน · ✕ ล้มเหลว` มีทั้งไอคอนและคำ ไม่ใช้สีอย่างเดียว

ไฟล์: `main.tsx` , `App.tsx` , `ScansPage.tsx` (ใหม่), `format.ts` (ใหม่), `types.ts` , `App.css`

10 แก้งาน 9 งานที่ค้าง "กำลังสแกน" ในฐานข้อมูล

ปัญหา

หน้าประวัติแสดงงาน #18–26 เป็น "กำลังสแกน" ตลอดไป เพราะเคยกด Ctrl+C ระหว่างสแกนตั้งแต่ก่อนมีระบบกู้งานค้าง (หนี้เทคนิคจากขั้นที่ 5)

ทำอย่างไร (ตัวเลือก A)

1. backup ก่อน: `pg_dump -Fc` ใน container แล้ว `docker cp` ออกมา (ไม่ใช้ `>` ของ PowerShell เพราะทำไฟล์ binary เสีย) → 2. ใน transaction เดียว: ScanRun 30 แถว และ ScanJob 9 แถว ตั้งเป็น `failed` พร้อมเหตุผล → 3. ตรวจตัวเลข → `COMMIT` · ผล: เสร็จ 11, ล้มเหลว 10, บางส่วน 1, กำลังสแกน 0

เรื่องที่เจอ

ครั้งแรกค้างเป็น session ที่ "idle in transaction" (ถือ lock ไว้) แก้ด้วยการปิด session นั้น · ต้นเหตุจริง (worker ไม่กู้งานที่ค้าง) ยังต้องแก้ใน C7

11 C5.4d ฟอร์มสั่งสแกน + ติดตามสถานะสด

```
[เริ่มสแกน] -POST /api/scans-> 202 {scan_job_id: 42}   (API ไม่รอให้สแกนเสร็จ)
     \-> ทุก 2 วินาที GET /api/scan-jobs/42 : รอคิว -> กำลังสแกน -> เสร็จ -> โหลดตารางประวัติใหม่
```

จุดสำคัญ

ส่วน	ทำไม
`setTimeout` ต่อกัน ไม่ใช้ `setInterval`	รอบใหม่เริ่มหลังรอบเก่าได้คำตอบแล้ว ถ้าเน็ตช้า request จะไม่ซ้อนกัน
หยุดเมื่อ completed / partial / failed	ไม่ถาม server ไปตลอดกาล
cleanup ใน `useEffect`	เปลี่ยนหน้าระหว่างสแกนแล้ว polling หยุด ไม่ค้างเบื้องหลัง
`useCallback` ที่ `refresh`	ไม่อย่างนั้นได้ฟังก์ชันใหม่ทุก render แล้ว effect ที่มีฟังก์ชันนี้ใน dependency จะเริ่มใหม่เรื่อยๆ จนวนไม่รู้จบ
`<fieldset disabled>`	ปิดทุกช่องและปุ่มระหว่างสแกน กันกดซ้ำ
ช่องกรอกตามที่ติ๊ก	SAST/Dependency ใช้โฟลเดอร์, Container ใช้ image, DAST ใช้ URL ตรงกับ `resolve_scans`
`aria-live="polite"`	โปรแกรมอ่านหน้าจออ่านสถานะใหม่ให้ฟัง
🔒 ไม่ตรวจ path/URL ในหน้าเว็บ	การตรวจที่ป้องกันได้จริงต้องอยู่ฝั่ง server (allowlist ใน C3) ลอง `C:\Windows` แล้วได้ "repo_path ไม่อยู่ในโฟลเดอร์ที่อนุญาต"

bug เล็กที่เจอตอนลองจริง

หลังส่งฟอร์มที่ถูกปฏิเสธ ข้อความ "งาน #42: กำลังตรวจสถานะ..." ยังค้างอยู่ เพราะล้างแค่ `job` ไม่ได้ล้าง `jobId` แก้ด้วย `setJobId(null)` ตอนเริ่มส่ง

ผลทดสอบ

งาน #42 สั่งจากเว็บ `triggered_by = web:admin` สถานะเปลี่ยนเองจนเสร็จ ตารางโหลดใหม่เอง

ไฟล์: `ScanForm.tsx` (ใหม่), `ScansPage.tsx`, `types.ts`, `App.css`

5. C5.5: ดาวน์โหลดและพิมพ์รายงาน

12 race condition: "เสร็จ" ก่อนรายงานพร้อม เจอจากการลองกับของจริง

อาการ

งาน #42 ขึ้น ✓ เสร็จ แต่คอลัมน์ report เป็น "-" ทั้งที่ worker สร้าง PDF แล้ว กดรีเฟรชจึงขึ้น "มี"

สาเหตุ

```
เดิม:  สแกนเสร็จ -> status = completed + commit   <- หน้าเว็บเห็นแล้วโหลดตารางทันที
       CWE -> EPSS -> สร้างรายงาน (หลายวินาที)  <- รายงานมาทีหลัง ตารางโหลดไปแล้ว
ใหม่:  สแกนเสร็จ -> ยังเป็น running
       CWE -> EPSS -> สร้างรายงาน -> status = completed + commit  <- ขั้นสุดท้าย
```

สองโปรแกรม (worker กับหน้าเว็บ) ทำงานพร้อมกัน และผลขึ้นกับว่าใครเร็วกว่า เรียกว่า **race condition**

แก้

`execute_scan_job()` คืนสถานะสุดท้ายแต่ยังไม่บันทึก · `run_full_pipeline()` บันทึกสถานะเป็นขั้นสุดท้าย · ครอบการสร้างรายงานด้วย `try/except` : ถ้ารายงานพัง (เช่นดิสก์เต็ม) ผลสแกนยังเป็น `completed` แค่ไม่มีไฟล์ แทนที่ทั้งงานจะกลายเป็น `failed`

test

แทนขั้นที่ช้าด้วยของปลอม (`monkeypatch`) แล้ว "แอบดู" สถานะตอนกำลังสร้างรายงาน ต้องเป็น `running` · ลองรัน test นี้กับโค้ดเดิมแล้วพังจริง (`assert 'running' == 'completed'`)

ผลทดสอบ

งาน #43 ขึ้นลิงก์รายงานทันทีที่เสร็จ โดยไม่ต้องรีเฟรช

ไฟล์: `save_findings.py` , `pipeline.py` , `tests/test_pipeline.py` (ใหม่, 2 test)

13 ดาวน์โหลด / เปิดดูเพื่อพิมพ์

backend: `?inline=true`

Content-Disposition	browser ทำอะไร
`attachment` (ค่าเริ่มต้น)	บันทึกเป็นไฟล์ `NodeGoat_scanjob43_security_report.pdf`
`inline`	เปิด PDF ในแท็บใหม่ กด Ctrl+P พิมพ์ได้

ส่วนที่ตรวจว่าไฟล์อยู่ใต้ `reports/` ทำงานเหมือนเดิมทั้ง 2 โหมด และมี test ยืนยันว่า path ที่ชี้ไป `.env` ได้ 404

frontend

ใช้ `<a href="/api/scan-jobs/43/report">` ธรรมดา เพราะ browser แนบ cookie ให้เองและจัดการดาวน์โหลดเอง · 🔒 `target="_blank"` ต้องคู่กับ `rel="noopener noreferrer"` ไม่ให้แท็บใหม่เข้าถึงหน้าเดิมผ่าน `window.opener` · ทดสอบ logout แล้วเปิดลิงก์ตรงๆ ได้ 401

เก็บกวาด

ชื่อแท็บเป็น **VulnCompass**, `lang="th"` ให้โปรแกรมอ่านหน้าจอออกเสียงถูก, ลบไฟล์ตัวอย่างของ Vite 4 ไฟล์

ไฟล์: `main.py` , `tests/test_report_file.py` (ใหม่, 3 test), `ScansPage.tsx` , `App.css` , `index.html`

6. C6a–C6b: CI ของ frontend และ security gate

14 C6a ตรวจ frontend บน GitHub ทุก push

```
push -> GitHub Actions -+-> [pytest]                                  (รันพร้อมกัน)
                        +-> [frontend] npm ci --ignore-scripts -> lint -> build
```

จุดสำคัญ

ส่วน	ทำไม
🔒 `permissions: contents: read`	token ใน workflow อ่านโค้ดได้อย่างเดียว ถ้า library ไหนเป็นมัลแวร์ก็แก้ repo ไม่ได้ (**least privilege**)
`npm ci` แทน `npm install`	ติดตั้งตาม lock เป๊ะ ไม่อัปเดตเอง lock ไม่ตรงก็ error ทันที
🔒 `--ignore-scripts`	ไม่รันสคริปต์ `postinstall` ของ library ซึ่งเป็นช่องทางโจมตี supply chain ที่พบบ่อย (ทดสอบแล้วว่า build ได้โดยไม่ต้องใช้)
`npm run build`	`tsc -b` ตรวจ type + `vite build` ลอง build จริง

ผล

`pytest` และ `frontend` เขียวทั้งคู่ · คำเตือน "chunk larger than 500 kB" มาจาก Recharts เป็นแค่คำเตือน

ไฟล์: `.github/workflows/tests.yml`

15 C6b `--fail-on` security gate

แนวคิด

CI ตัดสินผ่าน/ไม่ผ่านจาก **exit code** เท่านั้น (0 = ผ่าน) `--fail-on high` = ถ้ายังมีช่องโหว่ระดับ high ขึ้นไป (high + critical) เปิดอยู่ ให้จบด้วย exit 1 GitHub จะไม่ยอมให้ merge

`security_gate.py`

`severities_at_or_above()` ใช้ slice ของ `SEVERITY_ORDER` · `count_blocking_findings()` นับด้วย `IN` + `GROUP BY` · `gate_message()` เรียงผลจากร้ายแรงสุดเสมอ · ย้าย `SEVERITY_ORDER` มาไว้ที่นี่ที่เดียว ให้ API ใช้ร่วม (**single source of truth**)

กันพลาด

`choices=SEVERITY_ORDER` : พิมพ์ผิดเป็น `severe` จะ error ทันที แทนที่จะนับได้ 0 แล้ว**ผ่านไปเงียบๆ** (อันตรายกว่า) · ห้ามใช้คู่กับ `--queue` เพราะตอนจบคำสั่งยังไม่ได้สแกน · สแกนพัง → exit 1 ก่อนดู gate

ข้อควรรู้

นับช่องโหว่ที่ยังเปิดอยู่ทั้งโปรเจกต์ ไม่ใช่แค่จากเครื่องมือที่รันรอบนี้ ช่องโหว่เก่าที่ยังไม่แก้จึงยังทำให้ gate ไม่ผ่าน · ลองกับ NodeGoat จริง: `critical 22` → exit 1

test 8 ตัว

`monkeypatch.setattr(sys, "argv", [...])` จำลองการพิมพ์คำสั่ง · `pytest.raises(SystemExit)` จับว่า argparse ปฏิเสธจริง

ไฟล์: `security_gate.py` (ใหม่), `run_pipeline.py` , `main.py` , `tests/test_security_gate.py` (ใหม่)

7. C6c-1: รหัสผ่านฐานข้อมูลหลุดใน git ร้ายแรงที่สุดในรอบนี้

เจอได้อย่างไร: ระหว่างเตรียมให้ CI สร้างฐานข้อมูลใหม่ ต้องดูว่า alembic ได้ URL ฐานข้อมูลมาจากไหน จึงเจอว่า `alembic.ini` บรรทัด 89 มี `postgresql+psycopg2://secpipeline:<รหัสผ่าน>@localhost...` ตรวจโดยไม่แสดงค่าแล้วพบว่า**เป็นรหัสเดียวกับใน** `.env` และถูก commit ตั้งแต่ Phase 0 (`c2a8480`) จึงอยู่ในประวัติ git ทุก commit

ความเสี่ยงที่ซ้อนกัน	ผล
รหัสอยู่ในประวัติ git	ถ้าเปิด repo เป็น public เพื่อใช้เป็น portfolio ใครก็อ่านได้
Postgres `"5432:5432"`	เปิดรับทุก network card ใครอยู่ Wi-Fi เดียวกันก็ต่อได้
Redis `"6379:6379"` ไม่มีรหัส	ใครอยู่เครือข่ายเดียวกันก็ใส่งานสแกนเข้าคิวได้

16 แก้ 3 ชั้น

1. เอาออกจากโค้ด

`alembic/env.py` อ่าน `DATABASE_URL` จาก `db.py` (ซึ่งอ่าน `.env`) · แทน `%` เป็น `%%` เพราะ ini ใช้ `%` เป็นอักขระพิเศษ · `alembic.ini` เหลือ `sqlalchemy.url =`

2. ปิด port

`"127.0.0.1:5432:5432"` และ `"127.0.0.1:6379:6379"` แล้ว `docker compose up -d` สร้าง container ใหม่ ข้อมูลไม่หายเพราะอยู่ใน volume `pg_data`

3. เปลี่ยนรหัส (rotate)

เอาออกจากไฟล์อย่างเดียวไม่พอ เพราะรหัสเก่ายังอยู่ในประวัติ git · ไม่เลือกเขียนประวัติ git ใหม่ (ยุ่งยาก เสี่ยง และสำเนาที่คนอื่นมีก็ยังมีรหัสอยู่) แต่เปลี่ยนรหัสให้รหัสเก่า**ใช้ไม่ได้** · สร้างด้วย `secrets.token_urlsafe(24)` (32 ตัว ใช้ใน URL ได้โดยไม่ต้อง escape)

ผลตรวจสุดท้าย (ไม่แสดงรหัส)

รหัสใน `.env` ใช้ login ได้ ☑ · รหัสเก่าที่หลุดถูกปฏิเสธ ☑ · เก็บเป็น `scram-sha-256` ☑ · port เป็น `127.0.0.1` ทั้งคู่ ☑

ไฟล์: `alembic/env.py` , `alembic.ini` , `docker-compose.yml` , `.env` (ไม่อยู่ใน git)

ปัญหาระหว่างแก้ (บทเรียนจริง)

ปัญหา	สาเหตุ	แก้ / บทเรียน
`alembic current` พัง `UnicodeDecodeError`	**ผมสอนให้ใส่คอมเมนต์ภาษาไทยใน `alembic.ini`** แต่บน Windows alembic อ่าน .ini ด้วย encoding ของระบบ ไม่ใช่ UTF-8 · ตอนทดสอบในโฟลเดอร์ทดลองไม่ได้ใส่คอมเมนต์นั้น	เปลี่ยนเป็นภาษาอังกฤษ · ไฟล์ `.py` ใช้ไทยได้ (Python อ่าน UTF-8 เสมอ) แต่ไฟล์ config อื่นอาจไม่ได้ · **ทดสอบต้องใช้ไฟล์เดียวกับที่จะใช้จริงทุกตัวอักษร**
หลัง `\password` รหัสใน `.env` ใช้ไม่ได้	ช่องรหัสไม่แสดงตัวอักษร ค่าที่วางเข้าไปไม่ใช่รหัสที่ copy (เช่น Ctrl+V กลายเป็น `^V`) และผิดแบบเดียวกันทั้ง 2 ครั้ง psql จึงไม่เตือน	ตรวจแบบไม่แสดงรหัส: รหัสเก่าถูกปฏิเสธ (เปลี่ยนได้) แต่รหัสใน `.env` ก็ถูกปฏิเสธ (ค่าไม่ตรง) · แก้ด้วยสคริปต์ที่อ่าน `.env` แล้วส่ง `ALTER USER` เข้า psql ผ่าน **stdin** รหัสไม่ขึ้นจอ ไม่อยู่ในประวัติคำสั่ง ไม่ผ่านแชท
uvicorn/worker ต่อฐานข้อมูลไม่ได้หลังเปลี่ยนรหัส	จำรหัสเก่าไว้ตั้งแต่ตอนเปิด	restart ทั้งคู่ · `POSTGRES_PASSWORD` ใน compose ใช้แค่ตอนสร้างฐานข้อมูลครั้งแรก หลังจากนั้นต้องเปลี่ยนด้วย `\password` / `ALTER USER`

8. C6c-2: ให้ VulnCompass สแกนตัวเองบน GitHub

```
push / Pull Request -> GitHub เปิด Postgres ชั่วคราว (services)
                    -> Python 3.14, ฟอนต์ไทย, Trivy (ตรวจ SHA-256)
                    -> init_db.py -> seed CWE -> run_pipeline.py --repo . --fail-on critical
                    -> แนบ PDF ไว้ดาวน์โหลด (แนบทั้งตอนผ่านและไม่ผ่าน)
```

17 `init_db.py`: สร้างฐานข้อมูลใหม่จากศูนย์

ปัญหา

migration แรก (`ead95a4e9489`) ว่างเปล่า (`pass`) ตารางถูกสร้างครั้งแรกด้วยสคริปต์ทดลอง `alembic upgrade head` บนฐานข้อมูลใหม่จึงพัง

แก้

มีตาราง `alembic_version` → ฐานข้อมูลเดิม: `upgrade head` · ไม่มี → `Base.metadata.create_all()` สร้างทุกตารางจาก `models.py` แล้ว `stamp head` (เขียนป้ายว่าเป็นเวอร์ชันล่าสุด ไม่ได้รัน migration) · รันซ้ำกี่รอบก็ปลอดภัย (**idempotent**) ใช้ต่อใน Docker ตอน C7 ได้

18 สคริปต์ + workflow supply chain security

`scripts/install_trivy.sh`

ล็อกเวอร์ชัน 0.74.0 และ**เขียน SHA-256 ทางการไว้ในไฟล์** ถ้าไฟล์ที่โหลดมาถูกแก้ ค่าไม่ตรงแล้วหยุด · ไม่ใช้ `aquasecurity/trivy-action` เพราะเคยถูกแฮก tag มาแล้ว · `set -euo pipefail` หยุดทันทีเมื่อคำสั่งไหนพัง · `trap ... EXIT` ลบไฟล์ชั่วคราวเสมอ (คล้าย `finally`)

`scripts/ci_self_scan.sh`

ขั้นตอนสแกนอยู่ในสคริปต์เดียว ให้ทั้ง GitHub และ Docker ในเครื่องเรียกชุดเดียวกัน · `${FAIL_ON:-critical}` = `os.getenv("FAIL_ON", "critical")`

`.gitattributes`

`*.sh text eol=lf` แก้บน Windows แล้วยังเป็น LF ใน git เสมอ (Linux รันไฟล์ที่ขึ้นบรรทัดแบบ CRLF ไม่ได้)

`security-scan.yml`

ส่วน	ทำไม
`services: postgres` + `pg_isready`	ฐานข้อมูลชั่วคราวระหว่างรัน job รอให้พร้อมก่อนเริ่ม · รหัส `ci-only-throwaway` ไม่ใช่ความลับ เพราะอยู่ไม่กี่นาทีใน VM ที่ไม่มีใครเข้าถึง
🔒 `@<commit SHA> # v7.0.1`	tag ถูกเจ้าของย้ายไปชี้โค้ดอื่นได้ SHA เปลี่ยนไม่ได้
🔒 `persist-credentials: false`	ไม่เก็บ token ของ git ไว้ให้โค้ดของ scanner เอาไปใช้
`timeout-minutes: 30`	ค้างแล้วหยุดเอง ไม่กินโควตา 2,000 นาที/เดือน
`if: always()` ที่ upload	แนบรายงานแม้ gate ไม่ผ่าน ซึ่งเป็นตอนที่อยากดูมากที่สุด
`workflow_dispatch`	มีปุ่ม Run workflow ให้กดรันเอง

ไฟล์: `init_db.py` , `scripts/install_trivy.sh` , `scripts/ci_self_scan.sh` , `.gitattributes` , `.github/workflows/security-scan.yml` (ใหม่ทั้งหมด), `.github/workflows/tests.yml`

19 ทดสอบบนเครื่อง Windows ให้ใกล้ Linux ที่สุด

ทางเลือก

วิธี	ใกล้ของจริง	เลือกไหม
Docker รันขั้นตอนเดียวกัน	~90%	☑ ใช้ของที่มีอยู่ เร็ว
`act` (รันไฟล์ workflow ในเครื่อง)	~95%	❌ image หลาย GB ได้เพิ่มแค่นิดเดียว
push ขึ้น branch แล้วเปิด PR	100%	☑ ใช้เป็นรอบสุดท้าย

ผลใน Docker (python:3.14-slim + Postgres แยก ลบทิ้งหลังทดสอบ)

Trivy checksum OK · `init_db` สร้างตาราง + stamp ได้ และรันซ้ำแล้วเข้าทาง "ฐานข้อมูลเดิม" · CWE mapping 53 รายการ · Semgrep เจอ 6 (medium) · Trivy ตรวจ `requirements.txt` และ `frontend/package-lock.json` ได้ **0 ช่องโหว่** · PDF ภาษาไทยใช้ฟอนต์ Loma อ่านออก · gate ผ่าน exit 0

20 ผลบน GitHub + dogfooding

ระบบเจอปัญหาในตัวเอง

Semgrep เจอ "GitHub Actions step uses a mutable tag" ใน `tests.yml` 4 บรรทัด จึงล็อกด้วย SHA ในรอบเดียวกัน
ผลบน GitHub: Semgrep **6 → 2** (เหลือ jinja2 และ urllib ระดับ medium ที่ต้องตรวจต่อใน C7) ·
ตอนแรกผมคาดว่าจะเหลือ 5 เพราะนับผิดว่าเป็น 1 รายการ แต่จริงๆ แยกเป็นรายการละบรรทัด

Annotations รอบแรก

⚠ "Node.js 20 is deprecated" → อัปเกรด checkout v7.0.1, setup-python v7.0.0, setup-node v7.0.0, upload-artifact v7.0.1 (ไม่เลือก v7.1.0 / v7.0.2 เพราะออกมาไม่ถึง 2 สัปดาห์) · ตรวจแล้วว่า input ที่ใช้ยังมีครบ และ breaking change ไม่กระทบ · ⓘ "ubuntu-latest will migrate to Ubuntu 26 on October 19, 2026" → ล็อก `ubuntu-24.04` ไม่ให้ CI พังเองตอนชื่อแพ็กเกจเปลี่ยน · รอบที่ 2: ไม่มี Annotations เลย ใช้เวลา 1m 19s

Git workflow แบบทีม

`git switch -c ci-self-scan` → commit → `git push -u` → เปิด PR #1 → CI ตรวจ 3 ตัว → commit แก้เพิ่ม (PR ตรวจซ้ำเอง) → แก้ชื่อ PR → Merge → ลบ branch → `git pull` ที่ `main`

9. ปัญหาที่เจอระหว่างทาง และบทเรียน

อาการ	สาเหตุ	วิธีแก้ / บทเรียน
มี `package.json`, `node_modules` โผล่ที่ `Projects\`	รัน `npm install` ผิดโฟลเดอร์	ลบทิ้ง · npm และ `code src\...` รันที่ `\frontend>` ส่วน pip, pytest, uvicorn, git ของ Python รันที่ `\Projects>`
มีโฟลเดอร์ `Projects\src\` แปลกๆ	รัน `code src\...` จาก Projects	ลบทิ้ง · ดู breadcrumb บนแท็บ VS Code ว่าเป็น `frontend > src`
decorator หลุดไปติดฟังก์ชันอื่น (2 ครั้ง)	วางโค้ดใต้บรรทัด `@app.get`	เขียน `tests/test_routes.py` ให้จับได้อัตโนมัติ
Pylance ขีดเส้น import ทุกบรรทัด	เปิด VS Code ที่ `C:\Allprojects` ซึ่งมี settings บังคับ Python ของระบบ	เปิดด้วย `code .` จาก `C:\Allprojects\Projects`
login ได้ HTTP 500	Docker Desktop ปิด	เปิด Docker → `docker compose up -d` (ควรเป็น 503 จะแก้ใน C7)
งานค้าง "รอคิว"	worker ไม่ได้รัน / กด Ctrl+C ไปแล้ว	เปิด `python worker.py` ค้างไว้ใน terminal ที่ 3 · ฟอร์มมีข้อความเตือนให้เช็ก
แก้ `pipeline.py` แล้วไม่มีผล	worker ไม่มี `--reload` ใช้โค้ดเก่าในหน่วยความจำ	restart worker ทุกครั้งที่แก้โค้ดฝั่งสแกน
คำสั่งยาวกลายเป็น 2 คำสั่ง / ขึ้น `thon`	copy แล้วถูกตัดบรรทัดหรือตัวอักษรหาย	คำสั่งต้องอยู่บรรทัดเดียว · วางทีละบรรทัด · ส่งผลเป็นภาพแคปแทน copy ข้อความ
VS Code ขึ้นเส้นแดงทั้งที่ `tsc` ผ่าน	TypeScript server ยังจำไฟล์เก่า	Ctrl+Shift+P → TypeScript: Restart TS Server · ถ้าแท็บมี ● อย่ากด Ctrl+S ทับ ให้ Revert File
หามุม LF/CRLF ไม่เจอ	status bar อยู่แถบล่างสุด	Ctrl+Shift+P → Change End of Line Sequence · `.gitattributes` กันไว้อีกชั้น
ผมนับ Semgrep ผิด (5 แทน 2)	ไม่รู้ว่าแยกรายการละบรรทัด	ตัวเลขจากเครื่องมือคือความจริง

คำสั่งและแนวคิดใหม่

คำสั่ง / แนวคิด	ใช้เมื่อ
`npm run dev` / `npm run lint` / `npx tsc -b` / `npm run build`	รันหน้าเว็บ / ตรวจโค้ด / ตรวจ type / build จริง (ที่ `\frontend>`)
`npm install <pkg>@<ver> --save-exact`	ติดตั้งแบบล็อกเวอร์ชัน
`git add --dry-run`	ซ้อมดูว่าจะ add อะไร
`echo $LASTEXITCODE`	ดู exit code ของคำสั่งล่าสุดใน PowerShell
`python init_db.py`	สร้าง/อัปเดตฐานข้อมูล
`python run_pipeline.py ... --fail-on critical`	สแกนแบบมี security gate
`git switch -c <branch>` · `git push -u origin <branch>` · `git branch -d` · `git fetch --prune`	ทำงานบน branch แยก แล้วเก็บกวาดหลัง merge
`docker ps --format "{{.Names}} {{.Ports}}"`	ดูว่า port เปิดให้ใคร

race condition · N+1 query · debounce · polling · idempotent · least privilege · pin by SHA · secret rotation · dogfooding	แนวคิดที่ใช้ในรอบนี้

ลำดับเปิดระบบทุกวัน

#	คำสั่ง	ที่
1	เปิด Docker Desktop รอ Engine running	
2	`docker compose up -d`	`\Projects>`
3	`uvicorn main:app --reload`	`\Projects>` terminal 1
4	`npm run dev` → http://localhost:5173	`\frontend>` terminal 2
5	`python worker.py`	`\Projects>` terminal 3

10. สถานะงาน และสิ่งที่ยังเหลือ

ความคืบหน้า

ช่วง	สถานะ
ขั้นที่ 1–4 · C3 API/คิว · C4 login	เสร็จ
C5 React frontend	เสร็จ
C6 CI/CD	เสร็จ
C7 Production + hardening	ถัดไป
C8 Portfolio	ยังไม่ทำ

หนี้เทคนิค (แก้ใน C7)

เรื่อง	รายละเอียด
ควรทำ worker ไม่กู้งานค้าง	กด Ctrl+C ระหว่างสแกนแล้วงานค้าง `running` (เคยต้องแก้ #18–26 ด้วยมือ)
ควรทำ ฐานข้อมูลล่มได้ 500	ควรตอบ 503 พร้อมข้อความที่เข้าใจได้
ควรทำ Docker ทั้งระบบ	รันได้ด้วย `docker compose up` คำสั่งเดียว (มี `init_db.py` และฟอนต์ไทยที่ทดสอบแล้ว)
ควรทำ Semgrep 2 รายการ	jinja2 (autoescape) และ urllib (scheme) ตรวจว่าเป็นช่องโหว่จริงหรือ false positive
`API_KEY` ใน `.env` ว่าง	API แบบ key ตอบ 503 อยู่ (fail closed) ต้องสร้างด้วย `secrets.token_urlsafe(32)` ถ้าจะใช้
Redis ไม่มีรหัส	ปิด port เหลือ localhost แล้ว แต่ควรตั้งรหัสด้วย
ตัวจำกัด login / session	อยู่ในหน่วยความจำ / cookie เพิกถอนไม่ได้ ควรย้ายไป Redis
ค้น `_` ได้ทุกแถว	full-text search
เวลาในรายงานจาก CI เป็น UTC	ใส่เขตเวลาในรายงาน
อื่นๆ จากขั้นที่ 5	proxy client IP, path reports แบบ relative, `datetime.utcnow`, warning httpx, `is_repo_allowed` → `is_path_inside`

Commit ในรอบนี้

commit	ตอน	ข้อความ
`9c26164`	C5.1	Add React + TypeScript frontend with Vite proxy to the API
`d8304f4`	C5.2	Add login page and session check to the frontend
`eec9f00`	C5.3a	Add project summary endpoint for the dashboard
`72e5da3`	C5.3b	Add dashboard summary cards with project selector
`1c6ab39`	C5.3c	Add findings-by-tool bar chart with Recharts
`36d3903`	C5.3d1	Add escaped text search to the findings end point
`cf26ec8`	C5.3d2	Add findings table with search, filters and pagination
`c989cec`	C5.3	Fix severity badge colors and stop wrapping tool and CWE columns
`12add55`	C5.4a	Add scan job history endpoint and route mapping test
`8893c6c`	C5.4b	Add React Router with Dashboard and Scans pages
`bcca6e3`	C5.4c	Add scan history page and share date formatting helpers
`ba7dede`	C5.4d	Add scan form with live status polling
`bce1093`	C5.5a	Mark scan jobs finished only after the report is saved and add inline report view
`4c082e8`	C5.5b	Add report download and print links and rename the app tab

`e05fa29`	C6a	Run frontend lint and build in CI with read-only token
`4a95c5c`	C6b	Add --fail-on security gate to the CLI
`4114ea9`	C6c-1	Read the database URL from env in alembic and bind services to localhost
`ff0cea8`	C6c-2	Add self-scan workflow with a fresh database and pin actions by commit SHA
`8ed706c`	C6c-2	Upgrade actions to Node 24 versions and pin the runner to Ubuntu 24.04
`f24d090`	PR #1	Merge pull request #1 from jjjecop/ci-self-scan

ไฟล์ที่เปลี่ยนในรอบนี้

ไฟล์	สถานะ
`frontend/src/` : `api.ts` , `LoginPage.tsx` , `Dashboard.tsx` , `ToolChart.tsx` , `FindingsTable.tsx` , `ScansPage.tsx` , `ScanForm.tsx` , `types.ts` , `format.ts`	ใหม่ ประมาณ 890 บรรทัด TypeScript
`security_gate.py` , `init_db.py` , `scripts/install_trivy.sh` , `scripts/ci_self_scan.sh` , `.gitattributes` , `security-scan.yml`	ใหม่
`main.py`	แก้ summary, ค้นหา, scan-jobs, inline report
`pipeline.py` , `save_findings.py` , `run_pipeline.py`	แก้ race condition, `--fail-on`
`alembic/env.py` , `alembic.ini` , `docker-compose.yml`	แก้ **security** รหัสผ่านและ port
`tests.yml`	แก้ job frontend, SHA, Ubuntu 24.04
`tests/`	ใหม่ test_search, test_routes, test_pipeline, test_report_file, test_security_gate · รวม **102 test**

11. สรุปให้อ่าน: รอบนี้ทำอะไร ทำไม และได้อะไร

ทำอะไร

รอบนี้เปลี่ยน VulnCompass จาก "ระบบหลังบ้านที่ต้องใช้ผ่าน terminal หรือ API" เป็น **ระบบที่คนทั่วไปใช้งานได้ผ่านหน้าเว็บ** และ **ระบบที่ทำงานอัตโนมัติใน CI/CD** ฝั่งหน้าเว็บสร้างด้วย React + TypeScript มี 3 หน้า: login, Dashboard (การ์ดสรุป กราฟ ตารางค้นหา) และการสแกน (สั่งสแกน ดูสถานะสด ดาวน์โหลด/พิมพ์รายงาน) ฝั่ง CI/CD ทุกครั้งที่ push โค้ด GitHub จะรัน test ทั้ง Python และหน้าเว็บ และ**ใช้ VulnCompass สแกนโค้ดของตัวเอง** ถ้าเจอช่องโหว่ระดับ critical จะไม่ยอมให้ผ่าน

ทำไมต้องทำ

หน้าเว็บ: เป้าหมายตั้งแต่ขั้นที่ 5 คือระบบต้องใช้ได้ 3 ทาง (CLI, CI/CD, เว็บ) และต้องใช้เป็นผลงานสมัครงานได้ คนที่ดู portfolio อยากเห็นของที่กดใช้ได้จริง ไม่ใช่แค่โค้ด และ React + TypeScript เป็นทักษะ frontend ที่ตลาดต้องการมากที่สุด

CI/CD และ security gate: เครื่องมือสแกนช่องโหว่มีประโยชน์ที่สุดเมื่อมันทำงาน**ก่อน**โค้ดจะเข้า `main` (shift-left) ไม่ใช่หลังจากขึ้น production ไปแล้ว `--fail-on` ทำให้ผลสแกนกลายเป็นการตัดสินใจอัตโนมัติ ไม่ต้องมีคนมานั่งอ่านรายงานทุกครั้ง

แก้รหัสผ่านหลุด: โปรเจกต์ด้าน security ที่มีรหัสผ่านจริงอยู่ใน git จะเสียความน่าเชื่อถือทันทีถ้าคนอื่นเจอ การเจอเองและจัดการตามขั้นตอนที่ถูกต้อง (rotate, เอาออกจากโค้ด, ลดการเปิดเผย) เปลี่ยนจุดอ่อนให้กลายเป็นเรื่องที่เล่าได้

ได้อะไร

ด้าน	ผลที่ได้
ระบบ	ครบ 3 ช่องทาง: CLI (`run_pipeline.py`), CI/CD (GitHub Actions + `--fail-on`), เว็บ (login, dashboard, สแกน, รายงาน) ทุกช่องทางใช้แกนกลาง `run_full_pipeline()` ตัวเดียวกัน
ความปลอดภัยของระบบเอง	ไม่มีรหัสผ่านใน git, port เปิดเฉพาะ localhost, cookie HttpOnly, ค้นหาแบบ escape wildcard, ดาวน์โหลดได้เฉพาะไฟล์ใต้ `reports/` , CI ใช้ token อ่านอย่างเดียว, action ล็อกด้วย SHA, Trivy ตรวจ checksum
คุณภาพ	test 83 → 102 ตัว รวม test ที่**พิสูจน์แล้วว่าจับบั๊กจริงได้** (decorator หลุด, race condition) · CI 3 ตัวเขียวทั้งหมด · ESLint + TypeScript ไม่มีคำเตือน
ผลสแกนตัวเอง	Trivy: 0 ช่องโหว่ใน library ทั้ง Python และ npm · Semgrep: 2 รายการ medium (ลดจาก 6 เพราะแก้เรื่อง mutable tag ที่ระบบเจอเอง)
ทักษะ	React hooks (state, effect, cleanup, useCallback), TypeScript types, REST API design (N+1, GROUP BY, pagination), polling, GitHub Actions, Docker, Git branch + Pull Request

เรื่องที่เล่าตอนสัมภาษณ์งานได้

หัวข้อ	เล่าว่า
Secret incident response	เจอรหัสฐานข้อมูลใน git history → ประเมินความเสี่ยงที่ซ้อนกัน (port เปิด, Redis ไม่มีรหัส) → rotate แทนการเขียนประวัติใหม่ → ย้ายไปอ่านจาก env → ปิด port → ตรวจผลโดยไม่แสดงรหัส
Supply chain security	pin action by SHA, checksum ของ binary, `npm ci --ignore-scripts`, เลือกเวอร์ชันที่ออกมาเกิน 2 สัปดาห์, least-privilege token
Race condition	หน้าเว็บเห็น "เสร็จ" ก่อนรายงานพร้อม → ย้ายการบันทึกสถานะไปเป็นขั้นสุดท้าย → เขียน test ที่พังกับโค้ดเดิม
Shift-left + dogfooding	เครื่องมือของเราสแกนตัวเองใน CI และเจอปัญหาจริง (mutable tag) ที่แก้แล้วตัวเลขลดลงให้เห็น
Defense in depth	หน้าเว็บไม่ตรวจ path เอง ให้ server ตรวจด้วย allowlist และตรวจซ้ำก่อนส่งไฟล์รายงาน

> **สรุปในประโยคเดียว:** จบขั้นที่ 6 VulnCompass เป็นระบบสแกนช่องโหว่ที่ใช้ได้ทั้งจากเว็บ จาก terminal และใน CI ตรวจโค้ดของตัวเองทุกครั้งที่ push และผ่านการแก้ปัญหาด้าน security ของตัวเองจริงๆ มาแล้ว ขั้นต่อไป (C7) คือทำให้คนอื่นรันได้ด้วยคำสั่งเดียว และแก้หนี้เทคนิคที่เหลือ ก่อนทำ portfolio ใน C8